

LAPORAN PRAKTIKUM
PEMROGRAMAN WEB – GIK1JAB3
MODUL 12: MEMBUAT APLIKASI REACT BLOG POST MENGGUNAKAN API



Alya Mutiya Ningsih Pertiwi

707012500132

D4 SIKC 49-04

PROGRAM STUDI D4 SISTEM INFORMASI KOTA CERDAS

FAKULTAS ILMU TERAPAN

UNIVERSITAS TELKOM

BANDUNG

2026

1. Tujuan Praktikum

1. Memahami langkah-langkah integrasi antara frontend React dengan backend API (RESTful API).
2. Mampu melakukan instalasi dan konfigurasi proyek React menggunakan Vite.
3. Memahami dan menerapkan routing pada React menggunakan react-router-dom.
4. Mampu mengimplementasikan styling aplikasi React menggunakan framework Tailwind CSS.
5. Mampu melakukan manajemen state global, khususnya untuk autentikasi pengguna (Login/Logout) menggunakan React Context API.

2. Alat dan Bahan

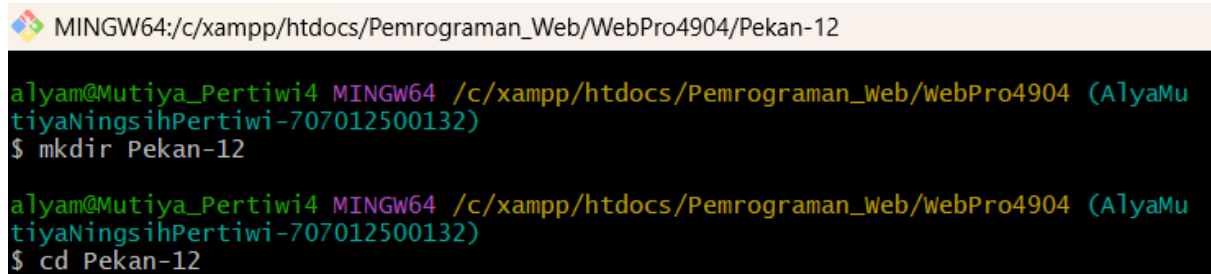
Perangkat Keras: Komputer/Laptop.

Perangkat Lunak:

- Web Browser (Google Chrome, Mozilla Firefox, dll).
- Code Editor (Visual Studio Code).
- Git Bash (sebagai terminal).
- Node.js & NPM (untuk inisiasi Vite dan instalasi paket frontend).
- XAMPP (Apache & MySQL untuk database API).
- Composer (untuk instalasi dependensi backend Laravel).

3. Langkah-langkah Praktikum

1. Buat folder baru di dalam /c/xampp/htdocs/Pemrograman_Web/WebPro4904:



```
MINGW64:/c/xampp/htdocs/Pemrograman_Web/WebPro4904/Pekan-12

alyam@Mutiya_Pertiwi4 MINGW64 /c/xampp/htdocs/Pemrograman_Web/webPro4904 (AlyaMu
tiyaNingsihPertiwi-707012500132)
$ mkdir Pekan-12

alyam@Mutiya_Pertiwi4 MINGW64 /c/xampp/htdocs/Pemrograman_Web/webPro4904 (AlyaMu
tiyaNingsihPertiwi-707012500132)
$ cd Pekan-12
```

2. Kemudian instalasi React menggunakan Vite:

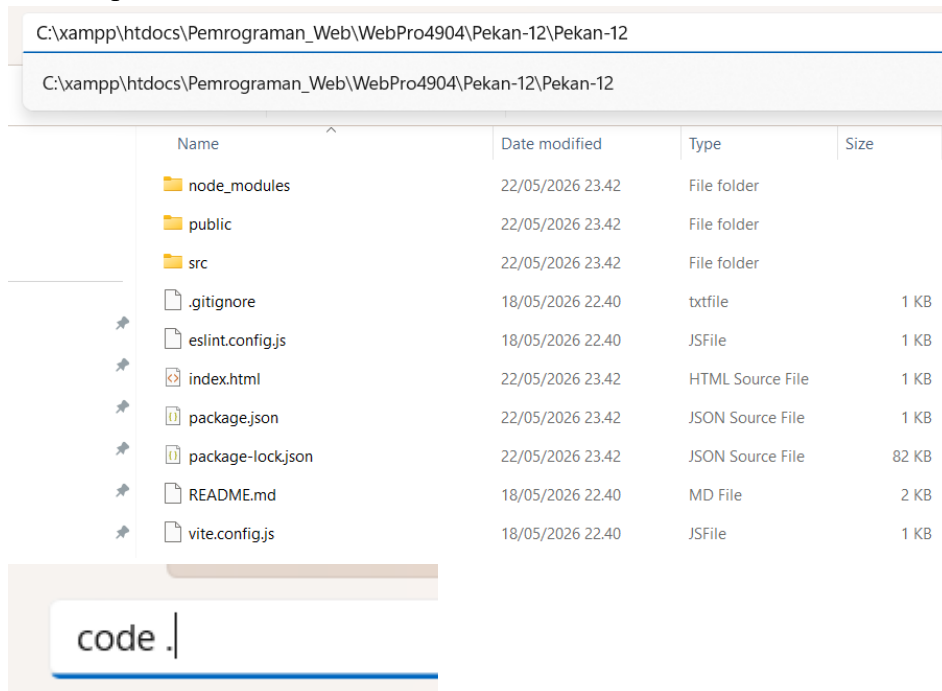
```
C:\windows\system32\cmd.exe
alyam@Mutiya_Pertiwi4 MINGW64 /c/xampp/htdocs/Pemrograman_Web/WebPro4904/Pekan-12 (AlyaMutiyaNingsihPertiwi-707012500132)
$ npm create vite@latest
> npx
> create-vite

o Project name:
  Pekan-12
o Package name:
  pekan-12
o Select a framework:
  React
o Select a variant:
  JavaScript
o Install with npm and start now?
  Yes
o Scaffolding project in C:\xampp\htdocs\Pemrograman_Web\WebPro4904\Pekan-12\Pekan-12...
o Installing dependencies with npm...

added 135 packages, and audited 136 packages in 9s
31 packages are looking for funding
  run 'npm fund' for details
found 0 vulnerabilities
o Starting dev server...
> pekan-12@0.0.0 dev
> vite

VITE v8.0.14 ready in 225 ms
➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

3. Kemudian buka folder yang sudah di buat dengan (code .), setelah itu folder akan terbuka pada VSCode:



4. Buka terminal pada vscode, dan ketik perintah “npm install axios react-router-dom” untuk menginstall react router dan juga axios:

```
PS C:\xampp\htdocs\Pemrograman_Web\WebPro4904\Pekan-12> cd Pekan-12
PS C:\xampp\htdocs\Pemrograman_Web\WebPro4904\Pekan-12\Pekan-12> npm install axios react-router-dom

added 29 packages, and audited 165 packages in 4s

38 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

5. Install Tailwind “npm install -D [tailwindcss@3.4.19](#) postcss@8.5.6 autoprefixer@10.4.24”:

```
PS C:\xampp\htdocs\Pemrograman_Web\WebPro4904\Pekan-12\Pekan-12> npm install -D tailwindcss@3.4.19 postcss@8.5.6 autoprefixer@10.4.24

added 61 packages, changed 1 package, and audited 226 packages in 5s

54 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

6. Generate file config (tailwind.config.js, postcss.config.js) “npx tailwindcss init -p”:

```
PS C:\xampp\htdocs\Pemrograman_Web\WebPro4904\Pekan-12\Pekan-12> npx tailwindcss init -p

Created Tailwind CSS config file: tailwind.config.js
Created PostCSS config file: postcss.config.js
```

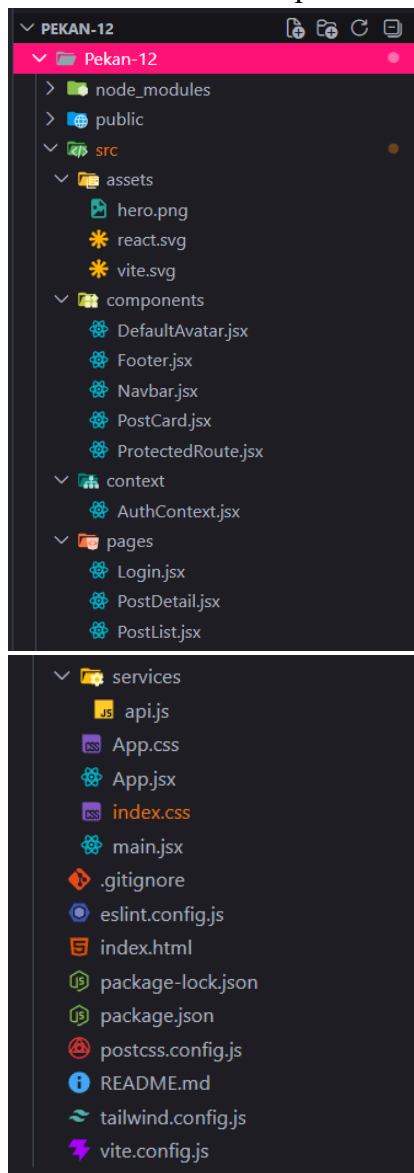
7. Buka file tailwind.config.js yang baru dibuat, lalu isi kode:

```
1  /** @type {import('tailwindcss').Config} */
2  export default {
3    content: [
4      "./index.html",
5      "./src/**/*.js,ts,jsx,tsx",
6    ],
7    darkMode: 'class',
8    theme: {
9      extend: {
10        colors: {
11          primary: {
12            50: '#eff6ff',
13            100: '#dbeafe',
14            200: '#bfdbfe',
15            300: '#93c5fd',
16            400: '#60a5fa',
17            500: '#3b82f6',
18            600: '#2563eb',
19            700: '#1d4ed8',
20            800: '#1e40af',
21            900: '#1e3a8a',
22          }
23        }
24      },
25    },
26    plugins: [],
27  }
28
```

8. Ubah isi index.css dengan kode berikut (untuk mengaktifkan Tailwind dan font default):

```
1  @tailwind base;
2  @tailwind components;
3  @tailwind utilities;
4
5  :root {
6    color-scheme: dark;
7  }
8
9  body {
10   font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
11 }
```

9. Buat struktur folder seperti berikut:



10. Pada file main.jsx dan import file App.jsx dan index.css isi file dengan kode berikut:

```
1 // Import ini digunakan untuk mengimpor beberapa modul dan komponen yang diperlukan dalam aplikasi.
2 // StrictMode diimpor dari react untuk membantu mendeteksi masalah potensial dalam aplikasi selama pengembangan.
3 // createRoot diimpor dari react-dom/client untuk membuat root DOM tempat aplikasi akan dirender.
4 // index.css diimpor untuk memberikan styling global pada aplikasi, dan App adalah komponen utama yang akan dirender sebagai bagian dari aplikasi.
5 import { StrictMode } from 'react'
6 import { createRoot } from 'react-dom/client'
7 import './index.css'
8 import App from './App.jsx'
9
10 // Fungsi createRoot digunakan untuk membuat root DOM tempat aplikasi akan dirender. Dengan menggunakan createRoot,
11 // dapat memanfaatkan fitur-fitur terbaru dari React, seperti concurrent mode, yang memungkinkan aplikasi untuk merender dengan lebih efisien dan responsif.
12 // Setelah root dibuat, memanggil metode render untuk merender komponen App di dalam StrictMode. StrictMode adalah alat pengembangan yang membantu mendeteksi masalah potensial dalam aplikasi,
13 // seperti penggunaan API yang sudah usang atau efek samping yang tidak diinginkan.
14 // Dengan membungkus App di dalam StrictMode, dapat memastikan bahwa aplikasi kita lebih stabil dan mudah untuk dipelihara selama pengembangan.
15 createRoot(document.getElementById("root")).render(
16   <StrictMode>
17     <App />
18   </StrictMode>,
19 )
```

11. Ubah file App.jsx dengan kode berikut:

```
1 // import ini digunakan untuk mengimpor beberapa modul dan komponen yang diperlukan dalam aplikasi.
2 // BrowserRouter, Routes, Route, dan Navigate diimpor dari react-router-dom untuk mengatur routing dalam aplikasi.
3 // AuthProvider diimpor dari context/authContext untuk menyediakan konteks autentikasi ke seluruh aplikasi. ProtectedRoute adalah komponen yang digunakan untuk melindungi rute tertentu agar hanya dapat diakses oleh pengguna yang sudah login.
4 // Login, PostList, dan PostDetail adalah halaman yang akan dirender berdasarkan rute yang ditentukan. Terakhir, index.css diimpor untuk memberikan styling global pada aplikasi.
5 import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
6 import { AuthProvider } from './context/AuthContext';
7 import ProtectedRoute from './components/ProtectedRoute';
8 import Login from './pages/login';
9 import PostList from './pages/PostList';
10 import PostDetail from './pages/PostDetail';
11 import './index.css';
12
13 // Fungsi App adalah komponen utama dari aplikasi yang mengatur struktur dan routing aplikasi. Di dalam fungsi ini, membungkus seluruh aplikasi dengan AuthProvider untuk menyediakan konteks autentikasi ke semua komponen di dalamnya.
14 // Router digunakan untuk mengatur routing, dan Routes digunakan untuk mendefinisikan rute-rute yang ada dalam aplikasi. Setiap Route memiliki path dan element yang menentukan komponen mana yang akan dirender ketika pengguna mengakses rute tersebut.
15 // ProtectedRoute digunakan untuk melindungi rute /posts dan /posts/:id agar hanya dapat diakses oleh pengguna yang sudah login. Jika pengguna mencoba mengakses rute yang tidak ada, mereka akan diarahkan ke halaman post.
16 // Fungsi App kemudian diekspor sebagai default export agar dapat digunakan di bagian lain aplikasi.
17 function App() {
18   return (
19     <AuthProvider>
20       <Router>
21         <Routes>
22           <Route path="/login" element={<Login />} />
23
24           <Route
25             path="/posts"
26             element={
27               <ProtectedRoute>
28                 <PostList />
29               </ProtectedRoute>
30             }
31           />
32
33           <Route
34             path="/posts/:id"
35             element={
36               <ProtectedRoute>
37                 <PostDetail />
38               </ProtectedRoute>
39             }
40           />
41
42           {/* Rute default untuk mengarahkan pengguna ke halaman post jika mereka mengakses root atau rute yang tidak ada.
43              Dengan menggunakan Navigate, memastikan bahwa pengguna akan diarahkan ke halaman yang benar jika mereka mencoba mengakses rute yang tidak valid. */}
44           <Route path="/" element={<Navigate to="/posts" replace />} />
45
46           {/* Rute wildcard untuk menangani semua rute yang tidak terdefinisi dan mengarahkan pengguna ke halaman posts.
47              Ini memastikan bahwa jika pengguna mencoba mengakses rute yang tidak ada, mereka akan diarahkan ke halaman utama daftar post,
48              memberikan pengalaman pengguna yang lebih baik dan mencegah kesalahan 404. */}
49           <Route path="*" element={<Navigate to="/posts" replace />} />
50         </Routes>
51       </Router>
52     </AuthProvider>
53   );
54 }
55
56 // Terakhir, mengexport fungsi App sebagai default export agar dapat digunakan di bagian lain aplikasi.
57 // Dengan menggunakan export default, kita memungkinkan fungsi ini untuk diimpor dengan nama apa pun di file lain,
58 // yang memberikan fleksibilitas dalam penggunaan komponen utama aplikasi ini di berbagai bagian aplikasi atau dalam pengujian.
59 export default App;
```

12. Isi file components/ProtectedRoute.jsx dengan kode berikut:

```
1 // Komponen ProtectedRoute digunakan untuk melindungi rute tertentu dalam aplikasi yang hanya boleh diakses oleh pengguna yang sudah terautentikasi.
2 // Komponen ini memeriksa status autentikasi pengguna melalui konteks AuthContext dan menampilkan konten yang sesuai berdasarkan status tersebut.
3 // Jika pengguna sedang memuat status autentikasi, komponen akan menampilkan indikator loading. Jika pengguna tidak terautentikasi, mereka akan diarahkan ke halaman login.
4 // Jika pengguna terautentikasi, komponen akan menampilkan konten anak yang diberikan sebagai prop.
5 import { Navigate } from 'react-router-dom';
6 import { useAuth } from '../context/AuthContext';
7
8 // Dalam komponen ProtectedRoute, kita menggunakan useAuth untuk mendapatkan status autentikasi pengguna dan status loading.
9 // Berdasarkan status ini, kita menentukan apa yang akan ditampilkan kepada pengguna.
10 // Jika aplikasi masih memuat status autentikasi, kita menampilkan indikator loading. Jika pengguna tidak terautentikasi, kita mengarahkan mereka ke halaman login. Jika
11 const ProtectedRoute = ({ children }) => {
12   const { isAuthenticated, loading } = useAuth();
13
14   // Bagian return dari komponen ProtectedRoute berisi logika untuk menentukan apa yang akan ditampilkan kepada pengguna berdasarkan status autentikasi dan loading.
15   // Jika aplikasi masih memuat status autentikasi, kita menampilkan indikator loading. Jika pengguna tidak terautentikasi, kita mengarahkan mereka ke halaman login menggunakan komponen Navigate dari react-router-dom.
16   // Jika pengguna terautentikasi, kita menampilkan konten anak yang diberikan sebagai prop.
17   if (loading) {
18     return (
19       <div className="loading-container">
20         <div className="loading-spinner"></div>
21         <p>loading...</p>
22       </div>
23     );
24   }
25
26   // Jika pengguna tidak terautentikasi, kita mengarahkan mereka ke halaman login menggunakan komponen Navigate dari react-router-dom.
27   // Properti replace digunakan untuk memastikan bahwa pengguna tidak dapat kembali ke halaman sebelumnya setelah diarahkan ke halaman login.
28   if (!isAuthenticated) {
29     return <Navigate to="/login" replace />;
30   }
31
32   // Jika pengguna terautentikasi, kita menampilkan konten anak yang diberikan sebagai prop.
33   // Konten ini akan menjadi komponen atau halaman yang ingin kita lindungi dengan rute ini.
34   return children;
35 };
36
37 // Ekspor komponen ProtectedRoute agar dapat digunakan di bagian lain aplikasi,
38 // seperti di App.jsx untuk melindungi rute tertentu yang hanya boleh diakses oleh pengguna yang sudah terautentikasi.
39 export default ProtectedRoute;
```

13. Isi file context/AuthContext.jsx dengan kode berikut:

```
1 // AuthContext.js adalah file yang berfungsi sebagai konteks untuk mengelola autentikasi pengguna dalam aplikasi React.
2 // Dalam file ini, kita membuat konteks menggunakan createContext dan menyediakan fungsi login dan logout yang dapat digunakan di seluruh aplikasi.
3 // Kita juga membuat provider, yaitu komponen yang menerima props children dan mengembalikan elemen JSX dengan status autentikasi yang ditentukan di localStorage.
4 // Selain itu, kita menggunakan useState untuk mengelola status autentikasi saat aplikasi diaktifkan dan memberikan status awal dengan informasi yang disimpan di localStorage.
5 // Dengan menggunakan AuthContext, kita dapat mengakses status autentikasi, informasi pengguna, dan fungsi login/logout di komponen mana pun dalam aplikasi.
6 import { createContext, useState, useEffect } from 'react';
7 import { authService } from './services/auth';
8
9 // Membuat konteks autentikasi dengan createContext, ini akan digunakan untuk menyediakan informasi autentikasi dan fungsi login/logout ke seluruh aplikasi.
10 const AuthContext = createContext(null);
11
12 // AuthProvider adalah komponen yang membungkus bagian aplikasi yang memerlukan akses ke konteks autentikasi.
13 // Di dalam AuthProvider, kita menerima props children dan mengembalikan elemen JSX dengan status autentikasi yang ditentukan di localStorage.
14 // Kita juga menggunakan useState untuk mengelola status autentikasi saat aplikasi diaktifkan dan memberikan status awal dengan informasi yang disimpan di localStorage.
15 export const AuthProvider = ({ children }) => {
16   const [user, setUser] = useState(null);
17   const [token, setToken] = useState(localStorage.getItem('token'));
18   const [loading, setLoading] = useState(true);
19
20   // useEffect digunakan untuk memeriksa status autentikasi saat aplikasi dimuat.
21   // Kita mencoba mengambil token dan informasi pengguna dari localStorage.
22   // Jika token ada, kita memverifikasi status autentikasi pengguna. Setelah memverifikasi berhasil, kita mengatur loading menjadi false untuk menunjukkan bahwa proses autentikasi telah selesai.
23   useEffect(() => {
24     const checkAuth = async () => {
25       const savedToken = localStorage.getItem('token');
26       const savedUser = localStorage.getItem('user');
27
28       // Jika token dan informasi pengguna ditemukan di localStorage,
29       // kita memverifikasi status autentikasi pengguna. Setelah memverifikasi berhasil, kita mengatur loading menjadi false untuk menunjukkan bahwa proses autentikasi telah selesai.
30       if (savedToken && savedUser) {
31         const { isAuthenticated } = authService.verifyToken(savedToken);
32         if (isAuthenticated) {
33           setUser(savedUser);
34           setLoading(false);
35         }
36       }
37
38       // Mengambil login checkAuth untuk memeriksa status autentikasi saat aplikasi dimuat.
39     };
40     checkAuth();
41   }, []);
42
43   // Fungsi login digunakan untuk mengirim permintaan login ke API dengan email dan password yang diberikan.
44   // Jika login berhasil, kita menyimpan token dan informasi pengguna di localStorage dan memverifikasi status.
45   // Jika login gagal, kita menangani error dan mengembalikan pesan error yang sesuai.
46   const login = async (email, password) => {
47     // Mengirim permintaan login dengan email dan password ke API menggunakan axios.
48     // Jika berhasil, kita menyimpan token dan informasi pengguna di localStorage dan memverifikasi status.
49     // Jika gagal, kita menangani error dan mengembalikan pesan error yang sesuai.
50     try {
51       const response = await authService.login(email, password);
52       const { access_token, data: userData } = response.data;
53
54       localStorage.setItem('token', access_token);
55       localStorage.setItem('user', JSON.stringify(userData));
56
57       setUser(userData);
58     } catch (error) {
59       // Jika terjadi error selama proses login, kita menangani error tersebut dan mengembalikan pesan error yang sesuai.
60       // Pesan error ini dapat digunakan di komponen login untuk memberikan umpan balik kepada pengguna tentang kegagalan login.
61     }
62   };
63
64   // Fungsi logout digunakan untuk menghapus sesi pengguna dengan mengirim API logout dan membersihkan token serta informasi pengguna dari localStorage dan state.
65   // Ini memastikan bahwa pengguna keluar dari aplikasi dan tidak dapat mengakses konten yang dilindungi tanpa login kembali.
66   const logout = async () => {
67     try {
68       await authService.logout();
69     } catch (error) {
70       console.error('Logout error:', error);
71     } finally {
72       localStorage.removeItem('token');
73       localStorage.removeItem('user');
74       setUser(null);
75     }
76   };
77
78   // Mengatur objek value yang berisi informasi pengguna, token, status loading, status autentikasi, dan fungsi login/logout.
79   // Objek ini akan disediakan ke seluruh aplikasi melalui AuthContext.Provider.
80   // Status autentikasi dihitung berdasarkan keberadaan token, sehingga jika token ada, isAuthenticated akan bernilai true, dan jika tidak ada, isAuthenticated akan bernilai false.
81   const value = {
82     user,
83     token,
84     loading,
85     isAuthenticated: !!token,
86     login,
87     logout,
88   };
89
90   // Mengembalikan AuthContext.Provider yang membungkus children dengan value yang telah dibuat.
91   // Ini memungkinkan komponen anak untuk mengakses informasi autentikasi dan fungsi login/logout melalui konteks ini.
92   return (
93     <AuthContext.Provider value={value}>
94       {children}
95     </AuthContext.Provider>
96   );
97 };
98
99 // Custom hook useAuth digunakan untuk memudahkan akses ke konteks autentikasi di komponen lain.
100 // Hook ini menerima useContext dengan AuthContext untuk mendapatkan nilai dari konteks.
101 // Jika hook ini digunakan di luar AuthProvider, kita melempar error untuk memberi tahu pengembang bahwa mereka harus menggunakan hook ini di dalam AuthProvider.
102 // Dengan menggunakan useAuth, kita dapat dengan mudah mengakses status autentikasi, informasi pengguna, dan fungsi login/logout di komponen mana pun dalam aplikasi tanpa harus secara eksplisit mengimpor dan menggunakan useContext setiap kali.
103 export const useAuth = () => {
104   const context = useContext(AuthContext);
105   if (!context) {
106     throw new Error('useAuth harus digunakan di dalam AuthProvider');
107   }
108   return context;
109 };
110
```

14. Isi file components/DefaultAvatar.jsx:

```
1 // Berfungsi sebagai avatar default ketika pengguna belum mengunggah foto profil. Komponen ini menggunakan SVG untuk menampilkan ikon pengguna standar.
2 const DefaultAvatar = ({ className = "w-7 h-7" }) => {
3   <div className={` ${className} rounded-full bg-gray-300 flex items-center justify-center flex-shrink-0`} >
4     <svg className="w-4 h-4 text-gray-500 fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
5       <path fillRule="evenodd" d="M10 9a3 3 0 10-6 3 3 0 006 3z" />
6     </svg>
7   </div>
8 };
9
10 // Ekspor komponen DefaultAvatar agar dapat digunakan di bagian lain aplikasi, seperti di Navbar.jsx untuk menampilkan avatar pengguna.
11 export default DefaultAvatar;
12
```


15. Isi file components/Navbar.jsx:

```
// Komponen Navbar yang berfungsi sebagai elemen untuk menampilkan informasi tentang situs web, seperti logo, nama merek, dan informasi kontak.
// Navbar ini dirancang dengan Tailwind CSS untuk memastikan responsivitas dan kemudahan penggunaan di berbagai ukuran layar.
// Elemen ini mencakup elemen-elemen berikut:
// 1. Logo situs web.
// 2. Nama merek.
// 3. Informasi kontak (alamat, telepon, email).
// 4. Informasi tentang produk atau layanan yang ditawarkan.
// 5. Informasi tentang lokasi fisik (alamat, peta).
// 6. Informasi tentang layanan yang ditawarkan.
// 7. Informasi tentang layanan yang ditawarkan.
// 8. Informasi tentang layanan yang ditawarkan.
// 9. Informasi tentang layanan yang ditawarkan.
// 10. Informasi tentang layanan yang ditawarkan.
// 11. Informasi tentang layanan yang ditawarkan.
// 12. Informasi tentang layanan yang ditawarkan.
// 13. Informasi tentang layanan yang ditawarkan.
// 14. Informasi tentang layanan yang ditawarkan.
// 15. Informasi tentang layanan yang ditawarkan.
// 16. Informasi tentang layanan yang ditawarkan.
// 17. Informasi tentang layanan yang ditawarkan.
// 18. Informasi tentang layanan yang ditawarkan.
// 19. Informasi tentang layanan yang ditawarkan.
// 20. Informasi tentang layanan yang ditawarkan.
// 21. Informasi tentang layanan yang ditawarkan.
// 22. Informasi tentang layanan yang ditawarkan.
// 23. Informasi tentang layanan yang ditawarkan.
// 24. Informasi tentang layanan yang ditawarkan.
// 25. Informasi tentang layanan yang ditawarkan.
// 26. Informasi tentang layanan yang ditawarkan.
// 27. Informasi tentang layanan yang ditawarkan.
// 28. Informasi tentang layanan yang ditawarkan.
// 29. Informasi tentang layanan yang ditawarkan.
// 30. Informasi tentang layanan yang ditawarkan.
// 31. Informasi tentang layanan yang ditawarkan.
// 32. Informasi tentang layanan yang ditawarkan.
// 33. Informasi tentang layanan yang ditawarkan.
// 34. Informasi tentang layanan yang ditawarkan.
// 35. Informasi tentang layanan yang ditawarkan.
// 36. Informasi tentang layanan yang ditawarkan.
// 37. Informasi tentang layanan yang ditawarkan.
// 38. Informasi tentang layanan yang ditawarkan.
// 39. Informasi tentang layanan yang ditawarkan.
// 40. Informasi tentang layanan yang ditawarkan.
// 41. Informasi tentang layanan yang ditawarkan.
// 42. Informasi tentang layanan yang ditawarkan.
// 43. Informasi tentang layanan yang ditawarkan.
// 44. Informasi tentang layanan yang ditawarkan.
// 45. Informasi tentang layanan yang ditawarkan.
// 46. Informasi tentang layanan yang ditawarkan.
// 47. Informasi tentang layanan yang ditawarkan.
// 48. Informasi tentang layanan yang ditawarkan.
// 49. Informasi tentang layanan yang ditawarkan.
// 50. Informasi tentang layanan yang ditawarkan.
// 51. Informasi tentang layanan yang ditawarkan.
// 52. Informasi tentang layanan yang ditawarkan.
// 53. Informasi tentang layanan yang ditawarkan.
// 54. Informasi tentang layanan yang ditawarkan.
// 55. Informasi tentang layanan yang ditawarkan.
// 56. Informasi tentang layanan yang ditawarkan.
// 57. Informasi tentang layanan yang ditawarkan.
// 58. Informasi tentang layanan yang ditawarkan.
// 59. Informasi tentang layanan yang ditawarkan.
// 60. Informasi tentang layanan yang ditawarkan.
// 61. Informasi tentang layanan yang ditawarkan.
// 62. Informasi tentang layanan yang ditawarkan.
// 63. Informasi tentang layanan yang ditawarkan.
// 64. Informasi tentang layanan yang ditawarkan.
// 65. Informasi tentang layanan yang ditawarkan.
// 66. Informasi tentang layanan yang ditawarkan.
// 67. Informasi tentang layanan yang ditawarkan.
// 68. Informasi tentang layanan yang ditawarkan.
// 69. Informasi tentang layanan yang ditawarkan.
// 70. Informasi tentang layanan yang ditawarkan.
// 71. Informasi tentang layanan yang ditawarkan.
// 72. Informasi tentang layanan yang ditawarkan.
// 73. Informasi tentang layanan yang ditawarkan.
// 74. Informasi tentang layanan yang ditawarkan.
// 75. Informasi tentang layanan yang ditawarkan.
// 76. Informasi tentang layanan yang ditawarkan.
// 77. Informasi tentang layanan yang ditawarkan.
// 78. Informasi tentang layanan yang ditawarkan.
// 79. Informasi tentang layanan yang ditawarkan.
// 80. Informasi tentang layanan yang ditawarkan.
// 81. Informasi tentang layanan yang ditawarkan.
// 82. Informasi tentang layanan yang ditawarkan.
// 83. Informasi tentang layanan yang ditawarkan.
// 84. Informasi tentang layanan yang ditawarkan.
// 85. Informasi tentang layanan yang ditawarkan.
// 86. Informasi tentang layanan yang ditawarkan.
// 87. Informasi tentang layanan yang ditawarkan.
// 88. Informasi tentang layanan yang ditawarkan.
// 89. Informasi tentang layanan yang ditawarkan.
// 90. Informasi tentang layanan yang ditawarkan.
// 91. Informasi tentang layanan yang ditawarkan.
// 92. Informasi tentang layanan yang ditawarkan.
// 93. Informasi tentang layanan yang ditawarkan.
// 94. Informasi tentang layanan yang ditawarkan.
// 95. Informasi tentang layanan yang ditawarkan.
// 96. Informasi tentang layanan yang ditawarkan.
// 97. Informasi tentang layanan yang ditawarkan.
// 98. Informasi tentang layanan yang ditawarkan.
// 99. Informasi tentang layanan yang ditawarkan.
// 100. Informasi tentang layanan yang ditawarkan.
```

```
1 <div className="flex items-center space-x-4">
2 <div className="hidden sm:flex items-center space-x-3">
3 <DefaultAvatar className="w-8 h-8" />
4 <span className="text-sm font-medium text-gray-900">
5 {user?.name || 'User'}
6 </span>
7 </div>
8 <button
9   onClick={handleLogout}
10  className="text-sm text-white bg-red-500 hover:bg-red-600 font-medium rounded-lg px-4 py-2 transition-colors"
11 >
12   Logout
13 </button>
14
15 <button
16   onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
17   className="md:hidden p-2 text-gray-600 hover:text-gray-900"
18 >
19   <svg className="w-6 h-6 fill="none" stroke="currentColor" viewBox="0 0 24 24">
20     <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M4 6h16M4 12h16M4 18h16" />
21   </svg>
22 </button>
23 </div>
24
25 /* Menu mobile yang muncul ketika tombol hamburger diklik.
26 Menu ini hanya terlihat pada perangkat dengan ukuran layar kecil (mobile) dan memberikan akses ke tautan navigasi yang sama seperti pada tampilan desktop.
27 Menu ini dirancang untuk memberikan pengalaman pengguna yang baik di perangkat mobile dengan menggunakan Tailwind CSS untuk styling dan responsivitas. */
28 <div className="md:hidden pb-4 border-t border-gray-200 mt-2 pt-4">
29 <div className="flex flex-col space-y-3">
30 <Link to="/posts" className="text-blue-600 font-medium">Home</Link>
31 <a href="#" className="text-gray-600 hover:text-gray-900">About</a>
32 <a href="#" className="text-gray-600 hover:text-gray-900">Contact</a>
33 </div>
34 </div>
35
36 </div>
37
38 </div>
39 </nav>
40
41 </div>
42
43 // Ekspor komponen Navbar agar dapat digunakan di bagian lain aplikasi, seperti di App.jsx untuk ditampilkan di seluruh halaman situs web.
44 export default Navbar;
```

16. Isi components/Footer.jsx:

```
// Komponen Footer yang berfungsi sebagai bagian bawah situs web, seperti informasi kontak, alamat, dan informasi lainnya.
// Footer ini dirancang dengan Tailwind CSS untuk memastikan responsivitas dan kemudahan penggunaan di berbagai ukuran layar.
// Elemen ini mencakup elemen-elemen berikut:
// 1. Informasi kontak (alamat, telepon, email).
// 2. Informasi tentang produk atau layanan yang ditawarkan.
// 3. Informasi tentang lokasi fisik (alamat, peta).
// 4. Informasi tentang layanan yang ditawarkan.
// 5. Informasi tentang layanan yang ditawarkan.
// 6. Informasi tentang layanan yang ditawarkan.
// 7. Informasi tentang layanan yang ditawarkan.
// 8. Informasi tentang layanan yang ditawarkan.
// 9. Informasi tentang layanan yang ditawarkan.
// 10. Informasi tentang layanan yang ditawarkan.
// 11. Informasi tentang layanan yang ditawarkan.
// 12. Informasi tentang layanan yang ditawarkan.
// 13. Informasi tentang layanan yang ditawarkan.
// 14. Informasi tentang layanan yang ditawarkan.
// 15. Informasi tentang layanan yang ditawarkan.
// 16. Informasi tentang layanan yang ditawarkan.
// 17. Informasi tentang layanan yang ditawarkan.
// 18. Informasi tentang layanan yang ditawarkan.
// 19. Informasi tentang layanan yang ditawarkan.
// 20. Informasi tentang layanan yang ditawarkan.
// 21. Informasi tentang layanan yang ditawarkan.
// 22. Informasi tentang layanan yang ditawarkan.
// 23. Informasi tentang layanan yang ditawarkan.
// 24. Informasi tentang layanan yang ditawarkan.
// 25. Informasi tentang layanan yang ditawarkan.
// 26. Informasi tentang layanan yang ditawarkan.
// 27. Informasi tentang layanan yang ditawarkan.
// 28. Informasi tentang layanan yang ditawarkan.
// 29. Informasi tentang layanan yang ditawarkan.
// 30. Informasi tentang layanan yang ditawarkan.
// 31. Informasi tentang layanan yang ditawarkan.
// 32. Informasi tentang layanan yang ditawarkan.
// 33. Informasi tentang layanan yang ditawarkan.
// 34. Informasi tentang layanan yang ditawarkan.
// 35. Informasi tentang layanan yang ditawarkan.
// 36. Informasi tentang layanan yang ditawarkan.
// 37. Informasi tentang layanan yang ditawarkan.
// 38. Informasi tentang layanan yang ditawarkan.
// 39. Informasi tentang layanan yang ditawarkan.
// 40. Informasi tentang layanan yang ditawarkan.
// 41. Informasi tentang layanan yang ditawarkan.
// 42. Informasi tentang layanan yang ditawarkan.
// 43. Informasi tentang layanan yang ditawarkan.
// 44. Informasi tentang layanan yang ditawarkan.
// 45. Informasi tentang layanan yang ditawarkan.
// 46. Informasi tentang layanan yang ditawarkan.
// 47. Informasi tentang layanan yang ditawarkan.
// 48. Informasi tentang layanan yang ditawarkan.
// 49. Informasi tentang layanan yang ditawarkan.
// 50. Informasi tentang layanan yang ditawarkan.
// 51. Informasi tentang layanan yang ditawarkan.
// 52. Informasi tentang layanan yang ditawarkan.
// 53. Informasi tentang layanan yang ditawarkan.
// 54. Informasi tentang layanan yang ditawarkan.
// 55. Informasi tentang layanan yang ditawarkan.
// 56. Informasi tentang layanan yang ditawarkan.
// 57. Informasi tentang layanan yang ditawarkan.
// 58. Informasi tentang layanan yang ditawarkan.
// 59. Informasi tentang layanan yang ditawarkan.
// 60. Informasi tentang layanan yang ditawarkan.
// 61. Informasi tentang layanan yang ditawarkan.
// 62. Informasi tentang layanan yang ditawarkan.
// 63. Informasi tentang layanan yang ditawarkan.
// 64. Informasi tentang layanan yang ditawarkan.
// 65. Informasi tentang layanan yang ditawarkan.
// 66. Informasi tentang layanan yang ditawarkan.
// 67. Informasi tentang layanan yang ditawarkan.
// 68. Informasi tentang layanan yang ditawarkan.
// 69. Informasi tentang layanan yang ditawarkan.
// 70. Informasi tentang layanan yang ditawarkan.
// 71. Informasi tentang layanan yang ditawarkan.
// 72. Informasi tentang layanan yang ditawarkan.
// 73. Informasi tentang layanan yang ditawarkan.
// 74. Informasi tentang layanan yang ditawarkan.
// 75. Informasi tentang layanan yang ditawarkan.
// 76. Informasi tentang layanan yang ditawarkan.
// 77. Informasi tentang layanan yang ditawarkan.
// 78. Informasi tentang layanan yang ditawarkan.
// 79. Informasi tentang layanan yang ditawarkan.
// 80. Informasi tentang layanan yang ditawarkan.
// 81. Informasi tentang layanan yang ditawarkan.
// 82. Informasi tentang layanan yang ditawarkan.
// 83. Informasi tentang layanan yang ditawarkan.
// 84. Informasi tentang layanan yang ditawarkan.
// 85. Informasi tentang layanan yang ditawarkan.
// 86. Informasi tentang layanan yang ditawarkan.
// 87. Informasi tentang layanan yang ditawarkan.
// 88. Informasi tentang layanan yang ditawarkan.
// 89. Informasi tentang layanan yang ditawarkan.
// 90. Informasi tentang layanan yang ditawarkan.
// 91. Informasi tentang layanan yang ditawarkan.
// 92. Informasi tentang layanan yang ditawarkan.
// 93. Informasi tentang layanan yang ditawarkan.
// 94. Informasi tentang layanan yang ditawarkan.
// 95. Informasi tentang layanan yang ditawarkan.
// 96. Informasi tentang layanan yang ditawarkan.
// 97. Informasi tentang layanan yang ditawarkan.
// 98. Informasi tentang layanan yang ditawarkan.
// 99. Informasi tentang layanan yang ditawarkan.
// 100. Informasi tentang layanan yang ditawarkan.
```

17. Isi components/PostCard.jsx:

```
1 // Komponen PostCard yang menampilkan ringkasan dari sebuah postingan artikel. Komponen ini menerima prop post yang berisi informasi tentang artikel, seperti judul, isi, penulis, dan tanggal pembuatan.
2 // PostCard dirancang untuk memberikan tampilan yang menarik dan informatif dengan menggunakan Tailwind CSS untuk styling. Komponen ini juga menyediakan tautan "Read more" yang mengarahkan pengguna ke halaman detail artikel ketika diklik.
3 import { Link } from "react-router-dom";
4 import DefaultAvatar from "../DefaultAvatar";
5
6 // Fungsi truncateText digunakan untuk memotong teks artikel jika melebihi batas karakter tertentu (default 150 karakter) dan menambahkan elipsis (...) di akhir teks yang dipotong.
7 // Fungsi ini memastikan bahwa ringkasan artikel tetap singkat dan mudah dibaca di dalam PostCard.
8 const PostCard = ({ post }) => {
9   const truncateText = (text, maxLength) => {
10     if (!text) return "";
11     if (text.length <= maxLength) return text;
12     return text.substring(0, maxLength) + "...";
13   };
14
15   // Fungsi formatDate digunakan untuk mengubah tanggal pembuatan artikel menjadi format yang lebih mudah dibaca, seperti "Today", "Yesterday", atau "X days ago".
16   // Fungsi ini menghitung selisih antara tanggal saat ini dan tanggal pembuatan artikel untuk menentukan format yang tepat.
17   const formatDate = (dateString) => {
18     if (!dateString) return "14 days ago";
19     const date = new Date(dateString);
20     const now = new Date();
21     const diffDays = Math.floor((now - date) / (1000 * 60 * 60 * 24));
22     if (diffDays === 0) return "Today";
23     if (diffDays === 1) return "Yesterday";
24     return `${diffDays} days ago`;
25   };
26
27   // Bagian return dari komponen PostCard berisi struktur JSX yang mendefinisikan tampilan dan tata letak kartu artikel.
28   // Tahap pertama adalah artikel, ringkasan isi, nama penulis, tanggal pembuatan, dan tautan "Read more".
29   // Kartu ini dirancang untuk memberikan informasi yang cukup tentang artikel tanpa menampilkan seluruh isi, sehingga pengguna dapat memutuskan apakah mereka ingin membaca lebih lanjut atau tidak.
30   return (
31     <article className="p-6 bg-white rounded-lg border border-gray-200 shadow-md hover:shadow-lg transition-shadow">
32       <div className="flex justify-between items-center mb-5 text-gray-500">
33         <span className="bg-blue-100 text-blue-800 text-xs font-medium inline-flex items-center px-2.5 py-0.5 rounded">
34           <svg className="w-1 h-3" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
35             <path fillRule="evenodd" d="M10 12 2 2 18 2 18 12" data-bbox="10 12 2 2 18 2 18 12" clipRule="evenodd"/>
36             <path d="M15 12 2 2 18 2 18 12" data-bbox="15 12 2 2 18 2 18 12" clipRule="evenodd"/>
37           </svg>
38           Article
39         </span>
40         <span className="text-sm flex items-center">
41           <svg className="w-4 h-4 mr-1" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
42             <path fillRule="evenodd" d="M10 2 2 10 12 18 20 10 18 2 10 2" data-bbox="10 2 2 10 12 18 20 10 18 2 10 2" clipRule="evenodd"/>
43           </svg>
44           <formatDate(post.created_at)>
45         </span>
46       </div>
47       <h2 className="mb-2 text-2xl font-bold tracking-tight text-gray-900">
48         <Link href={`/posts/${post.id}`} className="hover:text-blue-600 transition-colors">
49           <post.title>
50         </Link>
51       </h2>
52       <p className="mb-5 font-light text-gray-500">
53         <truncateText(post.article)>
54       </p>
55       <div className="flex justify-between items-center">
56         <div className="flex items-center space-x-3">
57           <DefaultAvatar />
58           <span className="font-medium text-gray-900">{post.author}</span>
59         </div>
60         <Link
61           href={`/posts/${post.id}`}
62           className="inline-flex items-center font-medium text-blue-600 hover:text-blue-500 transition-colors">
63           Read more
64         </div>
65         <div>
66           <svg className="w-4 h-4" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
67             <path fillRule="evenodd" d="M10 2 2 10 12 18 20 10 18 2 10 2" data-bbox="10 2 2 10 12 18 20 10 18 2 10 2" clipRule="evenodd"/>
68           </svg>
69         </div>
70       </div>
71     </article>
72   );
73
74 // Ekspor komponen PostCard agar dapat digunakan di bagian lain aplikasi, seperti di halaman utama yang menampilkan daftar artikel atau di halaman detail artikel.
75 export default PostCard;
```

18. Isi file pages/Login.jsx:

```
1 // Halaman login untuk memverifikasi identitas pengguna yang mencoba mengakses aplikasi. Halaman ini menerima prop user yang berisi informasi tentang pengguna, seperti email, password, dan status login.
2 // Halaman ini dirancang untuk memberikan tampilan yang menarik dan informatif dengan menggunakan Tailwind CSS untuk styling. Halaman ini juga menyediakan tautan "Forgot password" yang mengarahkan pengguna ke halaman reset password.
3 import { useState } from "react";
4 import { useRouter } from "react-router-dom";
5 import { authService } from "../services/authService";
6
7 // Komponen Login adalah fungsi yang menerima prop user yang berisi informasi tentang pengguna. Komponen ini menggunakan state untuk mengelola input email, password, dan status login.
8 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna. Komponen ini juga menerima prop user yang berisi informasi tentang pengguna.
9 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
10 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
11 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
12 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
13 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
14 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
15 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
16 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
17 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
18 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
19 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
20 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
21 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
22 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
23 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
24 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
25 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
26 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
27 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
28 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
29 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
30 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
31 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
32 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
33 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
34 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
35 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
36 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
37 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
38 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
39 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
40 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
41 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
42 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
43 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
44 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
45 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
46 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
47 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
48 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
49 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
50 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
51 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
52 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
53 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
54 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
55 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
56 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
57 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
58 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
59 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
60 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
61 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
62 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
63 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
64 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
65 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
66 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
67 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
68 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
69 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
70 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
71 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
72 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
73 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
74 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
75 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
76 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
77 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
78 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
79 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
80 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
81 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
82 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
83 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
84 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
85 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
86 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
87 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
88 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
89 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
90 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
91 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
92 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
93 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
94 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
95 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
96 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
97 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
98 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
99 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
100 // Fungsi ini juga menerima prop user yang berisi informasi tentang pengguna.
```

```

1 <div className="w-full max-w-md bg-white rounded-xl shadow-lg border border-gray-200 p-8">
2   <h1 className="text-2xl font-bold text-gray-900 mb-6">
3     Sign in to your account
4   </h1>
5
6   {error && (
7     <div className="mb-6 p-4 text-sm text-red-700 bg-red-100 rounded-lg border border-red-200 role="alert">
8       {error}
9     </div>
10  )}
11
12  <form className="space-y-6" onSubmit={handleSubmit}>
13    <div>
14      <label htmlFor="email" className="block mb-2 text-sm font-medium text-gray-900">
15        Your email
16      </label>
17      <input
18        type="email"
19        name="email"
20        id="email"
21        value={email}
22        onChange={e => setEmail(e.target.value)}
23        className="w-full px-4 py-3 bg-gray-50 border border-gray-300 rounded-lg text-gray-900 placeholder-gray-400 focus:ring-2 focus:ring-blue-500 focus:border-blue-500 outline-none transition-all"
24        placeholder="name@company.com"
25        required
26      />
27    </div>
28
29    <div>
30      <label htmlFor="password" className="block mb-2 text-sm font-medium text-gray-900">
31        Password
32      </label>
33      <input
34        type="password"
35        name="password"
36        id="password"
37        value={password}
38        onChange={e => setPassword(e.target.value)}
39        placeholder="password"
40        className="w-full px-4 py-3 bg-gray-50 border border-gray-300 rounded-lg text-gray-900 placeholder-gray-400 focus:ring-2 focus:ring-blue-500 focus:border-blue-500 outline-none transition-all"
41        required
42      />
43    </div>
44

```

```

1
2      <div className="flex items-center justify-between">
3        <div className="flex items-center">
4          <input
5            id="remember"
6            type="checkbox"
7            className="w-4 h-4 border border-gray-300 rounded bg-gray-50 focus:ring-2 focus:ring-blue-500"
8          />
9          <label htmlFor="remember" className="ml-2 text-sm text-gray-600">
10            Remember me
11          </label>
12        </div>
13        <a href="#" className="text-sm font-medium text-blue-600 hover:text-blue-500">
14          Forgot password?
15        </a>
16      </div>
17
18      <button
19        type="submit"
20        disabled={loading}
21        className="w-full px-4 py-3 bg-blue-600 hover:bg-blue-700 text-white font-medium rounded-lg transition-colors focus:ring-4 focus:ring-blue-300 disabled:opacity-50 disabled:cursor-not-allowed"
22      >
23        {loading ? 'Signing in...' : 'Sign in'}
24      </button>
25
26      <p className="text-sm text-gray-600 text-center">
27        Don't have an account yet?{' '}
28        <a href="#" className="font-medium text-blue-600 hover:text-blue-500">
29          Sign up
30        </a>
31      </p>
32    </form>
33  </div>
34
35  <footer className="bg-white border-t border-gray-200 py-4">
36    <p className="text-center text-sm text-gray-500">
37      © 2024 Blog Posts. All rights reserved.
38    </p>
39  </footer>
40 </div>
41
42 };
43
44 // Ekspor komponen Login agar dapat digunakan di bagian lain aplikasi, seperti di App.jsx untuk ditampilkan sebagai halaman login ketika pengguna mengakses rute yang memerlukan autentikasi.
45 export default Login;

```

19. Isi file components/PostDetail.jsx:

```
1 //import ini digunakan untuk membawa fungsi dan komponen yang diperlukan dari pustaka React, react-router-dom, dan layanan API yang telah dibuat sebelumnya.
2 import { useState, useEffect } from 'react';
3 import { useParams, Link } from 'react-router-dom';
4 import { postsAPI } from '../services/api';
5 import Navbar from '../components/Navbar';
6 import Footer from '../components/Footer';
7 import DefaultAvatar from '../components/DefaultAvatar';
8
9 // Komponen PostDetail adalah halaman yang menampilkan detail dari sebuah postingan artikel.
10 // Komponen ini menggunakan useParams untuk mendapatkan ID artikel dari URL, dan useState serta useEffect untuk mengelola state dan efek samping dalam mengambil data artikel dari API.
11 // Ketika komponen dimuat, useEffect memanggil fungsi fetchPost untuk mengambil data artikel berdasarkan ID yang diperoleh dari URL.
12 // Jika data berhasil diambil, state post akan diperbarui dengan data tersebut. Jika terjadi error, state error akan diperbarui dengan pesan error yang sesuai.
13 // Komponen ini juga menyediakan fungsi formatDate untuk mengubah tanggal pembuatan artikel menjadi format yang lebih mudah dibaca.
14 // Bagian return dari komponen ini berisi struktur JSX yang mendefinisikan tampilan halaman detail artikel, termasuk judul, isi, penulis, tanggal pembuatan, dan tautan untuk kembali ke daftar posts.
15 const PostDetail = () => {
16   const { id } = useParams();
17   const [post, setPost] = useState(null);
18   const [loading, setLoading] = useState(true);
19   const [error, setError] = useState('');
20
21   // useEffect digunakan untuk memanggil fungsi fetchPost ketika komponen pertama kali dimuat atau ketika ID artikel berubah.
22   useEffect(() => {
23     fetchPost();
24   }, [id]);
25
26   // Fungsi fetchPost digunakan untuk mengambil data artikel dari API berdasarkan ID yang diperoleh dari URL.
27   // Fungsi ini mengatur status loading menjadi true sebelum memulai proses pengambilan data.
28   // Jika data berhasil diambil, state post akan diperbarui dengan data tersebut dan pesan error akan diabaikan.
29   // Jika terjadi error, kita memeriksa apakah error tersebut adalah 404 (artikel tidak ditemukan) atau error lainnya, dan memperbarui state error dengan pesan yang sesuai.
30   // Setelah proses selesai, status loading akan diatur menjadi false.
31   const fetchPost = async () => {
32     try {
33       setLoading(true);
34       const response = await postsAPI.getById(id);
35       setPost(response.data.data);
36       setError('');
37     } catch (err) {
38       if (err.response?.status === 404) {
39         setError('Post tidak ditemukan.');
```


20. Isi pages/PostList.jsx:

```

1 // bagian return dari komponen PostList berisi string yang mendefinisikan tampilan halaman daftar posts.
2 // Ini mencakup Navbar di bagian atas, judul dan disclaimer singkat tentang blog, serta kondisi untuk menampilkan spinner loading, pesan error, atau daftar posts.
3 // Jika status loading true, akan ditampilkan spinner animasi. Jika ada pesan error, itu akan ditampilkan dengan tombol untuk mencoba lagi. Jika tidak ada posts yang tersedia, akan ditampilkan pesan yang menyatakan bahwa belum ada posts.
4 // Jika data posts berhasil diambil, kita menggunakan metode map untuk merender setiap post menggunakan komponen PostCard. Di bagian bawah halaman, footer akan ditampilkan untuk memberikan informasi tambahan atau navigasi.
5 // Halaman ini dirancang untuk menampilkan pengalaman pengguna yang baik dengan menggunakan Tailwind CSS untuk styling dan responsivitas.
6 return (
7   <div className="min-h-screen bg-gray-100 flex col">
8     <Navbar />
9
10    <main className="flex-1">
11      <div className="py-8 px-4 mx-auto max-w-screen-xl lg:py-16 lg:px-0">
12        <div className="mx-auto max-w-screen-sm text-center lg:mb-16 mb-8">
13          <h2 className="mb-4 text-xl lg:text-2xl tracking-tight font-extrabold text-gray-900">
14            Our Blog
15          </h2>
16          <p className="font-light text-gray-500 sm:text-xl">
17            Explore our collection of articles about React, web development, and more.
18          </p>
19        </div>
20
21        <loading ? (
22          <div className="flex justify-center items-center py-20">
23            <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div>
24          </div>
25        ) : error ? (
26          <div className="text-center py-20">
27            <p className="text-red-500 mb-4">{error}</p>
28            <button
29              onClick={fetchPosts}
30              className="text-white bg-blue-600 hover:bg-blue-700 font-medium rounded-lg text-sm px-5 py-2.5">
31              Coba Lagi
32            </button>
33          </div>
34        ) : posts.length === 0 ? (
35          <div className="text-center py-20">
36            <p className="text-gray-500">Belum ada posts.</p>
37          </div>
38        ) : (
39          <div className="grid gap-8 lg:grid-cols-2">
40            {posts.map((post) => (
41              <PostCard key={post.id} post={post} />
42            ))}
43          </div>
44        )}
45      </div>
46    </main>
47  </div>
48  <Footer />
49 </div>
50 )};
51
52 // Terakhir, kita mengimpor komponen PostList agar dapat digunakan di bagian lain aplikasi.
53 // Dengan menggunakan export default, kita memastikan komponen ini dapat diimpor dengan nama apa pun di file lain, meskipun biasanya kita akan mengimpor dengan nama yang sama untuk menjaga konsistensi.
54 export default PostList;

```

21. Isi file services/api.js, dimana API_URL harus disesuaikan dengan link localhost:

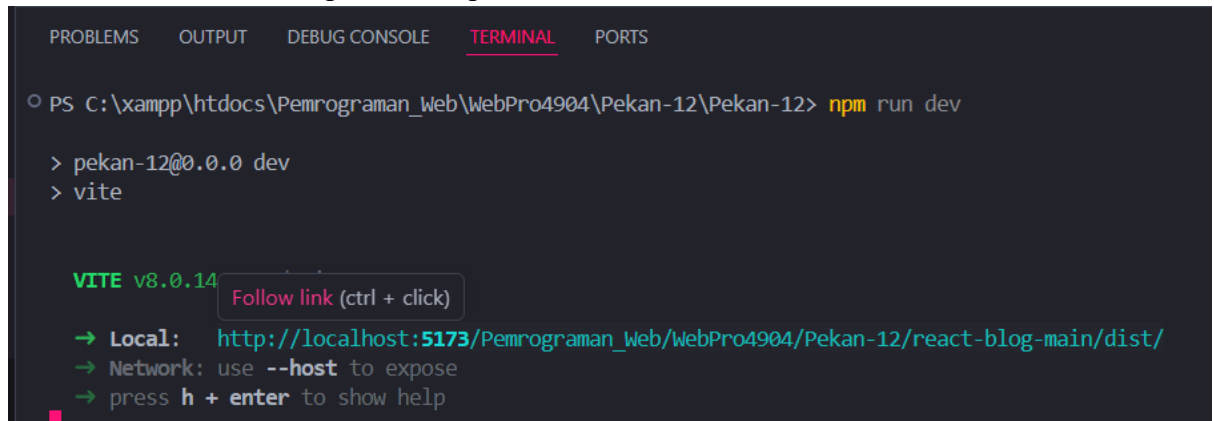
```
1 // import axios digunakan untuk membuat instance API yang akan digunakan untuk berkomunikasi dengan backend.
2 import axios from "axios";
3
4 // API_URL adalah URL dasar untuk API backend yang akan digunakan dalam aplikasi. Dalam contoh ini, URL mengarah ke server lokal yang menjalankan API.
5 const API_URL = "http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api";
6
7 // Kita membuat instance API menggunakan axios.create, yang memungkinkan kita untuk mengatur konfigurasi dasar seperti baseURL dan headers yang akan digunakan untuk semua permintaan API.
8 const api = axios.create({
9   baseURL: API_URL,
10  headers: {
11    "Content-Type": "application/json",
12    Accept: "application/json",
13  },
14 });
15
16 // Interceptor request digunakan untuk menambahkan token autentikasi ke header setiap permintaan API jika token tersebut tersedia di localStorage.
17 // Ini memastikan bahwa pengguna yang sudah login dapat mengakses endpoint yang memerlukan autentikasi tanpa harus menambahkan token secara manual setiap kali melakukan permintaan.
18 api.interceptors.request.use(
19   (config) => {
20     const token = localStorage.getItem("token");
21     if (token) {
22       config.headers.Authorization = `Bearer ${token}`;
23     }
24     return config;
25   },
26   (error) => {
27     return Promise.reject(error);
28   },
29 );
30
31 // Interceptor response digunakan untuk menangani kasus ketika token autentikasi tidak valid atau sudah kedaluwarsa.
32 // Jika server merespons dengan status 401 (Unauthorized), interceptor ini akan memeriksa URL permintaan.
33 // Jika URL tidak termasuk endpoint login atau register, maka token akan dihapus dari localStorage, dan pengguna akan diarahkan ke halaman login.
34 // Ini membantu menjaga keamanan aplikasi dengan memastikan bahwa pengguna yang tidak sah tidak dapat mengakses sumber daya yang dilindungi.
35 api.interceptors.response.use(
36   (response) => response,
37   (error) => {
38     if (error.response && error.response.status === 401) {
39       const requestUrl = error.config.url || "";
40
41       if (!requestUrl.includes("/login") && !requestUrl.includes("/register")) {
42         localStorage.removeItem("token");
43         localStorage.removeItem("user");
44         window.location.href = "/login";
45       }
46     }
47     return Promise.reject(error);
48   },
49 );
```

```
1 // authAPI adalah objek yang berisi metode untuk berinteraksi dengan endpoint autentikasi di API backend.
2 // Metode login digunakan untuk mengirim permintaan POST ke endpoint /auth/login dengan email dan password pengguna.
3 // Metode register digunakan untuk mengirim permintaan POST ke endpoint /auth/register dengan nama, email, password, dan konfirmasi password pengguna.
4 // Metode logout digunakan untuk mengirim permintaan POST ke endpoint /auth/logout untuk keluar dari sesi pengguna.
5 // Metode me digunakan untuk mengirim permintaan GET ke endpoint /auth/me untuk mendapatkan informasi tentang pengguna yang sedang login.
6 export const authAPI = {
7   login: (email, password) => api.post("/auth/login", { email, password }),
8   register: (name, email, password, password_confirmation) =>
9     api.post("/auth/register", { name, email, password, password_confirmation }),
10  logout: () => api.post("/auth/logout"),
11  me: () => api.get("/auth/me"),
12 };
13
14 // postsAPI adalah objek yang berisi metode untuk berinteraksi dengan endpoint posts di API backend.
15 // Metode getAll digunakan untuk mengirim permintaan GET ke endpoint /post dengan parameter page dan per_page untuk mendapatkan daftar posts dengan pagination.
16 // Metode getById digunakan untuk mengirim permintaan GET ke endpoint /post/{id} untuk mendapatkan detail dari post tertentu berdasarkan ID-nya.
17 export const postsAPI = {
18   getAll: (page = 1, perPage = 10) => api.get(`/post?page=${page}&per_page=${perPage}`),
19   getById: (id) => api.get(`/post/${id}`),
20 };
21
22 // Dengan struktur ini, kita memiliki satu file api.js yang mengelola semua interaksi dengan backend API.
23 export default api;
```

22. Isi file vite.config.js sebagai berikut, dimana '/ci3_project' harus disesuaikan dengan folder aplikasi API CI3 dari modul 9:

```
1 // Import ini digunakan untuk mengimport beberapa modul dan plugin yang diperlukan dalam konfigurasi Vite. defineConfig adalah fungsi dari Vite untuk membantu dalam mendefinisikan konfigurasi Vite dengan cara yang lebih terstruktur. Hasil diimport dari file ini.
2 import { defineConfig } from "vite"
3 import react from "@vitejs/plugin-react"
4
5 // https://vite.dev/config/
6 // Konfigurasi Vite diimport menggunakan defineConfig untuk memberikan struktur yang lebih baik dan dukungan untuk fitur file Vite. Dalam konfigurasi ini,
7 // kita mendefinisikan beberapa opsi untuk konfigurasi Vite.
8 // Menambahkan plugin react untuk mendukung pengembangan aplikasi React dengan vite. Selain itu, kita mengatur server proxy untuk memudahkan permintaan API ke backend Laravel yang berjalan di localhost, sehingga kita dapat menghindari masalah CORS selama pengembangan.
9 export default defineConfig({
10   base: "/Pemrograman_Web/WebPro4904/Pekan-12/react_blog/assets/dist/",
11   plugins: [react()],
12   server: {
13     // Konfigurasi proxy yang digunakan untuk mengarahkan permintaan API yang ditujukan ke "/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project" ke target "http://localhost".
14     // Dengan menggunakan changeOrigin: true, kita memastikan bahwa header Origin dalam permintaan akan diubah agar sesuai dengan target, yang membantu dalam menghindari masalah CORS selama pengembangan.
15     // Ini memungkinkan aplikasi React untuk berkomunikasi dengan backend Laravel tanpa mengalami masalah lintas domain.
16     proxy: {
17       "/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project": {
18         target: "http://localhost",
19         // Dengan mengatur target ini, kita memastikan bahwa semua permintaan yang ditujukan ke "/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project" akan diteruskan ke backend Laravel yang sesuai.
20         changeOrigin: true,
21       },
22     },
23   },
24 });
```

23. Buka terminal lalu ketik perintah “npm run dev”



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

○ PS C:\xampp\htdocs\Pemrograman_Web\WebPro4904\Pekan-12\Pekan-12> npm run dev

> pekan-12@0.0.0 dev
> vite

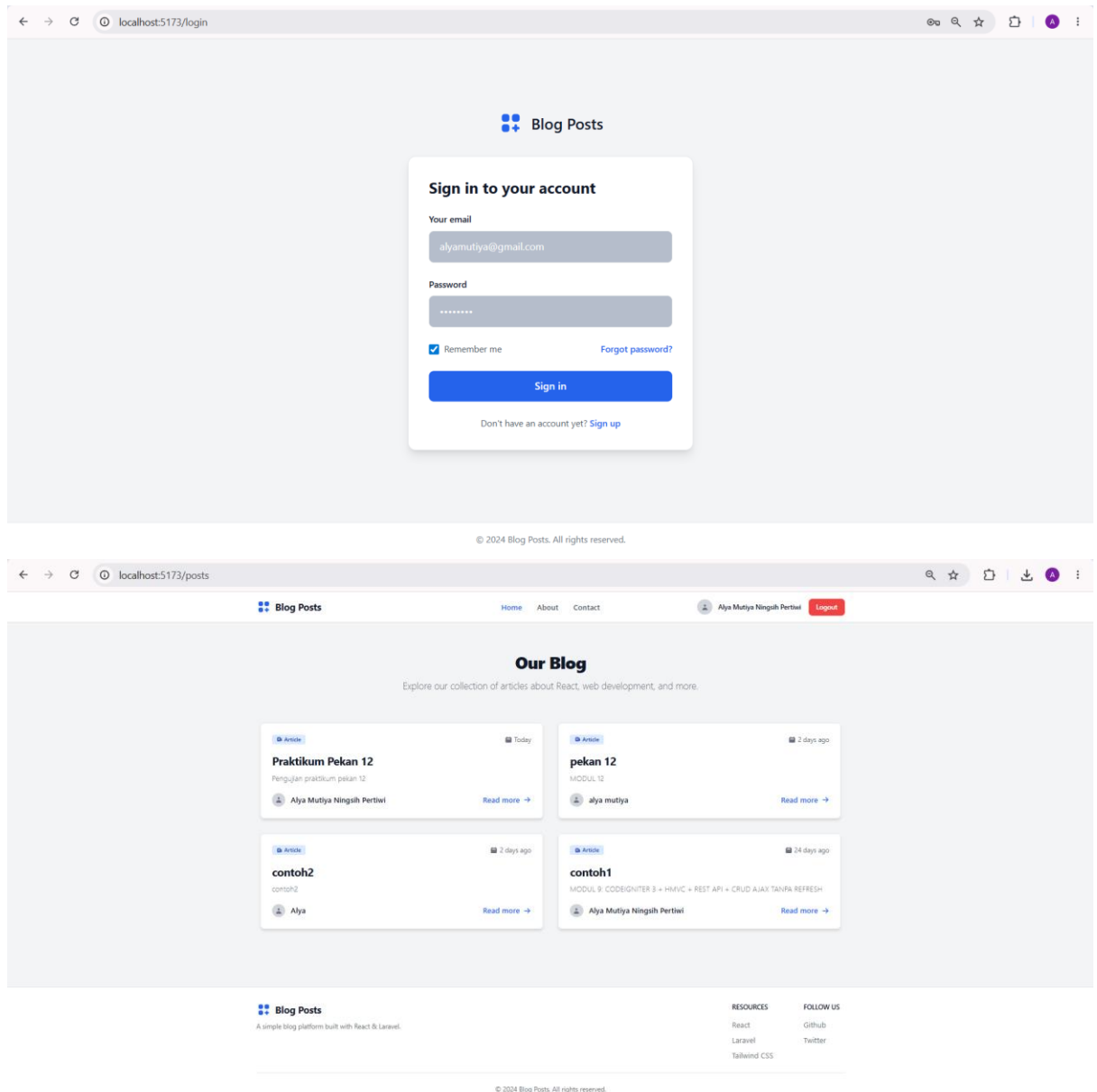
VITE v8.0.14
  Follow link (ctrl + click)
→ Local:   http://localhost:5173/Pemrograman_Web/WebPro4904/Pekan-12/react-blog-main/dist/
→ Network: use --host to expose
→ press h + enter to show help
```

24. Kemudian salin link yang terdapat pada terminal dan tampilkan hasilnya melalui chrome.

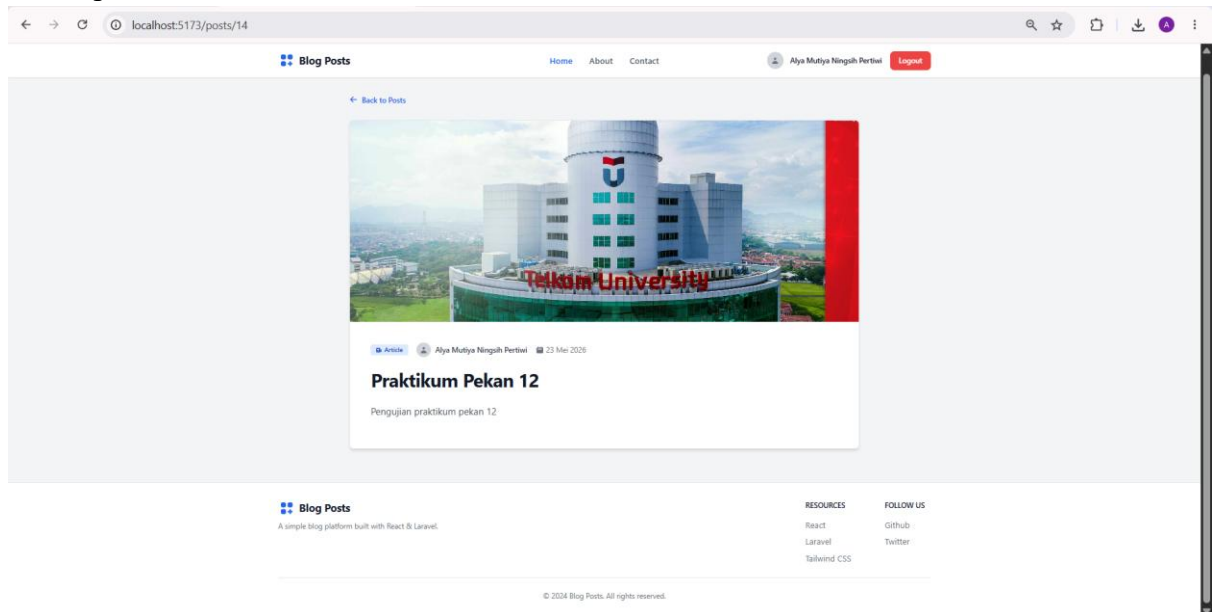
25. Selesai.

4. Hasil dan Dokumentasi Program

Login menggunakan email dan password yang telah dibuat pada praktikum Pekan-09



Detail post



5. Analisis dan Pembahasan

Pada praktikum ini, telah dibangun sebuah aplikasi Blog Post berbasis Single Page Application (SPA) menggunakan library React. Aplikasi ini tidak beroperasi secara sendiri, melainkan terintegrasi dengan REST API sebagai penyedia data utama.

- Pengembangan Frontend: Proyek diinisialisasi menggunakan Vite untuk performa kompilasi yang lebih cepat. Modul antarmuka (user interface) dibangun secara responsif menggunakan Tailwind CSS, yang memungkinkan penulisan style secara fungsional langsung di dalam file .jsx.
- Manajemen Rute & Akses: Navigasi antar halaman (seperti halaman Daftar Post, Detail Post, dan Login) dikelola oleh react-router-dom. Keamanan akses client-side diimplementasikan dengan komponen ProtectedRoute, yang memblokir akses ke halaman blog jika pengguna belum terautentikasi (belum memiliki token) dan mengarahkannya kembali ke halaman login.
- Integrasi API: Pengambilan data dan proses autentikasi ditangani menggunakan library Axios. Konfigurasi Axios (interceptors) diatur agar selalu menyimpan token Bearer di setiap header permintaan (request) jika pengguna sudah berhasil login.

6. Kesimpulan

Melalui praktikum pekan 12 ini, dapat disimpulkan bahwa React dipadukan dengan Vite dan Tailwind CSS yang sangat efisien untuk membangun antarmuka web modern. Pemisahan arsitektur antara frontend (React) dan backend (API) membuat pengembangan aplikasi menjadi lebih modular. Penggunaan fitur React seperti useState, useEffect untuk fetching data, Context API untuk manajemen status login, serta interceptors pada Axios sangat krusial dalam menciptakan aplikasi web yang dinamis, terpusat, dan aman.

7. Lampiran berisi Link repositori GitHub

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/AlyaMutiyaNingsihPertwi-707012500132>